LANGCHAIN CORE CONCEPTS

Understanding Runnables in LangChain

Beginner Friendly Guide

FROM MODELS & PROMPTS TO A UNIFIED INTERFACE

01 · THE CORE IDEA

What is a Runnable?

Building intuition before any code

SIMPLE DEFINITION

A Runnable is anything that can take an input, perform a task, and return an output.

That's it. No AI magic, no special model — just three steps. Anything that follows this same three-step shape can be treated as a Runnable, no matter what it does inside.

Real-Life Example: The Coffee Machine

WHY THIS IS A RUNNABLE

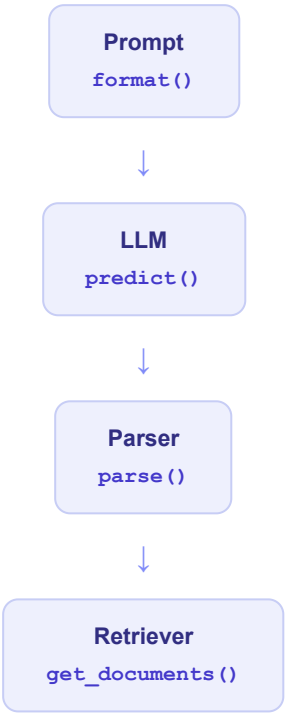
The machine takes an input (beans), performs one clear task (brewing), and produces an output (coffee). It doesn't know or care what happens before or after it — it just does its one job reliably. That is exactly the behaviour LangChain wants from every component: prompts, models, parsers, and retrievers.

02 · WHY THEY WERE INTRODUCED

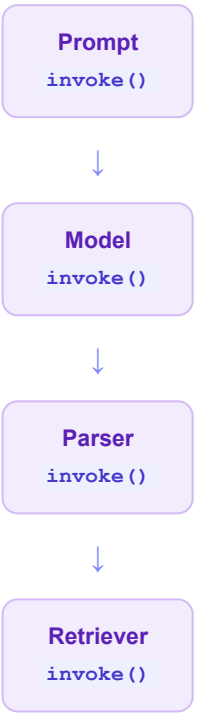
The Problem Before Runnables

Every component spoke a different language

✕ Before Runnable



✓ After Runnable



Before Runnables, every component had its own method name. Developers had to memorise a different API for every single piece — and large applications quickly became hard to maintain.

Old LangChain	Modern LangChain (Runnable)
Different method for every component	One shared method: `invoke()`
Hard to remember many APIs	Learn one interface, use it everywhere
Components don't connect naturally	Components snap together like pipes
Difficult to scale large apps	Clean, predictable, scalable structure

WHY THIS MATTERS

Giving every component the same shape means a developer who learns `invoke()` once already knows how to run *any* LangChain component — a prompt, a model, a parser, or a retriever.

03 · REAL WORLD ANALOGIES

Thinking About Runnables in Everyday Life

Two analogies that make the idea click

Analogy 1 — The Restaurant

Customer → Waiter → Chef → Cashier → Delivery Boy

Every person in this chain performs exactly **one** task. Each one receives an input, processes it, and passes on an output — the waiter takes the order, the chef cooks it, the cashier bills it, the delivery boy delivers it. Every person behaves like a Runnable.

Analogy 2 — The Factory Assembly Line

Raw Material → Machine 1 → Machine 2 → Machine 3 → Finished Product

Each machine performs one specific task on the item passed to it. No single machine builds the whole product alone — but together, the chain of machines completes the entire workflow. This is precisely how a LangChain pipeline works.

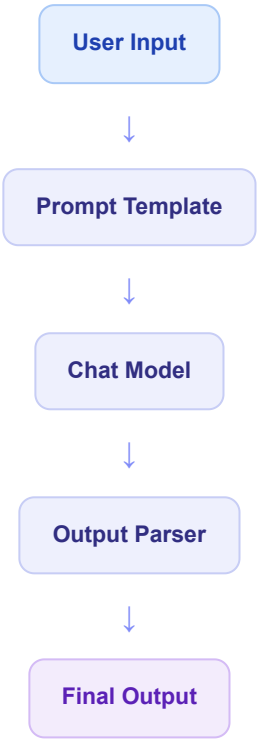
THE PATTERN TO NOTICE

In both analogies, nobody needs to know how the *other* steps work internally. They just need to agree on how to hand work to each other. That shared agreement is what the Runnable interface provides.

04 · CONNECTING THE PIECES

How Runnables Work Together

Why Prompt, Model, and Parser can now be connected



Every block in this flow — the prompt, the model, and the parser — is a Runnable. Because they all share the exact same shape (input in, output out), LangChain can connect them directly, one after another, with no extra glue code in between.

THE RUNNABLE PIPELINE

A pipeline is simply several Runnables connected in a row, where each one's output becomes the next one's input — just like the factory assembly line from the last page.

The Pipe Symbol (|)

A SIMPLE WAY TO THINK ABOUT IT

The `|` symbol just means "then." Prompt, then Model, then Parser. It works because every piece on either side of it is a Runnable — no technical detail needed to understand *why* it's allowed to connect this way.

05 · RUNNABLE TYPES

Common Runnable Types in LangChain

The building blocks you'll meet most often

1. RunnableLambda

Turns a normal Python function into a Runnable.

Example: a custom step that shortens or cleans up text.

2. RunnableSequence

Executes multiple Runnables one after another.

Example: Prompt → Model → Parser, in order.

3. RunnableParallel

Runs multiple Runnables at the same time.

Example: translating and summarising the same text together.

4. RunnableBranch

Runs different paths based on a condition (if–else).

Example: short messages take one path, long ones take another.

5. RunnablePassthrough

Passes the original input to the next step, unchanged.

Example: keeping the user's raw question available later in the chain.

6. Prompt Templates

Also a Runnable — it takes variables in and returns a formatted prompt out.

Example: filling "Translate: {text}" with a real sentence.

7. Chat Models

Also a Runnable — it takes a prompt in and returns a response out.

Example: the model answering a question.

8. Output Parsers

Also a Runnable — it takes a raw response in and returns clean data out.

Example: converting a model's reply into plain text.

9. Retrievers

Also a Runnable — it takes a query in and returns matching documents out.

Example: fetching relevant notes for a question.

GOOD TO KNOW

LangChain contains many more Runnable implementations than this. These nine are simply the ones beginners run into first — and understanding them covers almost everything you'll need early on.

06 · YOUR FIRST CODE

Simple Code Examples

Only beginner-friendly, minimal code

Example 1 — RunnableLambda

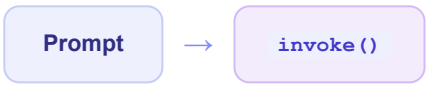

```
from langchain_core.runnables import RunnableLambda

# wrap a normal function so LangChain can run it
square = RunnableLambda(lambda x: x * x)

square.invoke(4)    # Output: 16
```

We took an ordinary function and wrapped it. Now it can `invoke()` just like every other Runnable.

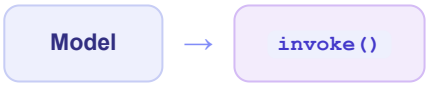
Example 2 — Prompt Runnable




```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Translate to French: {text}")
prompt.invoke({"text": "Good morning"})
```

Example 3 — Chat Model Runnable




```
from langchain_openai import ChatOpenAI

model = ChatOpenAI()
model.invoke("What is the capital of France?")
```

NOTICE THE PATTERN

Three completely different components — a function, a prompt, and a model — but every single one is run the exact same way: `.invoke()`.

07 · THE PAYOFF

Why Are Runnables Better?

Putting it all side by side

⊘ Without Runnable	✔ With Runnable
Different APIs for every component	One shared interface
Hard to connect components	Easy, direct connection
Repeated glue code	Reusable, composable pieces
Difficult maintenance	Clean pipeline architecture
Hard debugging	Cleaner, more predictable code
Doesn't scale well	Modern LangChain standard
Limited to small scripts	Scalable AI applications

WHY COMPANIES PREFER THIS DESIGN

Teams building real products need code that is predictable and easy to extend. When every component obeys the same interface, new engineers can understand an unfamiliar pipeline in minutes, testing becomes simpler, and swapping one piece (say, changing the model) never breaks the rest of the chain.

One Interface

Learn once, use everywhere

Composable

Snap pieces together freely

Scalable

Grows with your application

08 · WRAP UP

Summary & Student Quiz

Check your understanding

Key Takeaways

- ✔ A Runnable is **not** an AI model.
- ✔ A Runnable is a common interface shared across components.
- ✔ Every Runnable accepts an input and returns an output.
- ✔ Prompts, Models, Parsers, and Retrievers are all Runnables.
- ✔ Runnables connect easily to form pipelines.
- ✔ This is why LangChain offers `invoke()`, `batch()`, `stream()`, and the pipe operator.

Quiz — Test Yourself (Conceptual, No Coding)

1. In one sentence, what is a Runnable?
2. What three steps describe how every Runnable works?
3. Why did old LangChain become hard to maintain?
4. What single method replaced `format()`, `predict()`, and `parse()`?
5. In the restaurant analogy, what does the "waiter" represent?
6. Why can a Prompt, a Model, and a Parser be connected directly?
7. What does the pipe symbol `|` mean in a Runnable pipeline?
8. What does RunnableParallel do differently from RunnableSequence?
9. When would you use RunnablePassthrough?
10. Why do companies prefer building with the Runnable design?

REMEMBER THIS ABOVE ALL

Runnable is not a new AI model. It is simply a common interface that allows all LangChain components to work together.